



Signed Representation: Two's Complement

CONTENTS

Learning Outcomes

Two's Complement

Arithmetic and conversion

Formal definition

Two's Complement: Flip sign

Two's Complement: C standard (as of 2025)

1 Learning Outcomes

- Translate between decimal numbers and two's complement representations
- Compare two's complement to other signed and unsigned representations



Lecture Video



2 Two's Complement

Ones' complement:

- Problem: Negative mappings “overlap” with the positive ones, creating the two 0s.
- The solution: **Shift the negative mappings left by one.**

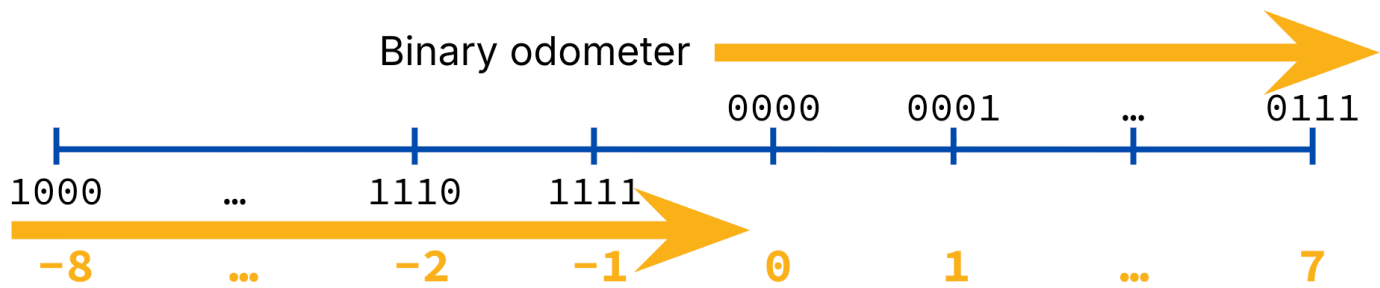


Figure 1: "Binary odometer" for 4-bit two's complement.

Of note:

- Like in ones' complement, incrementing the binary odometer corresponds with integer addition by one.
- `0b0000` is still 0.
- Positive numbers are the same as ones' complement.
- Negative numbers are shifted over! For example, `0b1111` now maps to -1. This gives us one extra negative number.
- The most significant bit (leftmost bit) can still be interpreted as the **sign bit**.

In Two's Complement, a bit pattern of all ones is -1.

Quick Check

Suppose you use N bits to represent integers with two's complement. How many positive numbers? negative numbers? zero?

 Show Answer



3 Arithmetic and conversion

Hardware for two's complement is now simple.

Addition is exactly the same as with an unsigned number.

The numbers 5 and -5 are represented in 4-bit two's complement with `0b0101` and `0b1011`, respectively. Adding them together should result in 0, or `0b0000`.



Figure 2: Addition in two's complement follows the decimal intuition.

Explanation

4 Formal definition

We can write the value of an n -digit two's complement number as

$$-2^{n-1}d_{n-1} + \sum_{i=0}^{n-2} 2^i d_i$$

Positive and negative numbers can be computed using the same formula. Above, the sign is computed by multiplying the highest bit by (-2^{N-1}) .

Example: 5 and -5 in 4-bit two's complement

$$\begin{aligned} 0b1011 &= (1 \times -2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ &= -8 + 0 + 2 + 1 \\ &= -5 \end{aligned}$$

$$\begin{aligned} 0b0101 &= (0 \times -2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 0 + 4 + 0 + 1 \\ &= 5 \end{aligned}$$

5 Two's Complement: Flip sign

Hardware to convert positive to negative (& vice versa) is simple.

1. Complement all bits
2. Then add 1



Figure 3: Two's Complement: To change sign, flip the bits and add one.

At home: Prove algorithm is equivalent to formula!

The intuition comes from a “number wheel” representation of our binary odometer (Figure [Figure 4](#)). This wheel also helps us understand identify where integer overflow occurs:

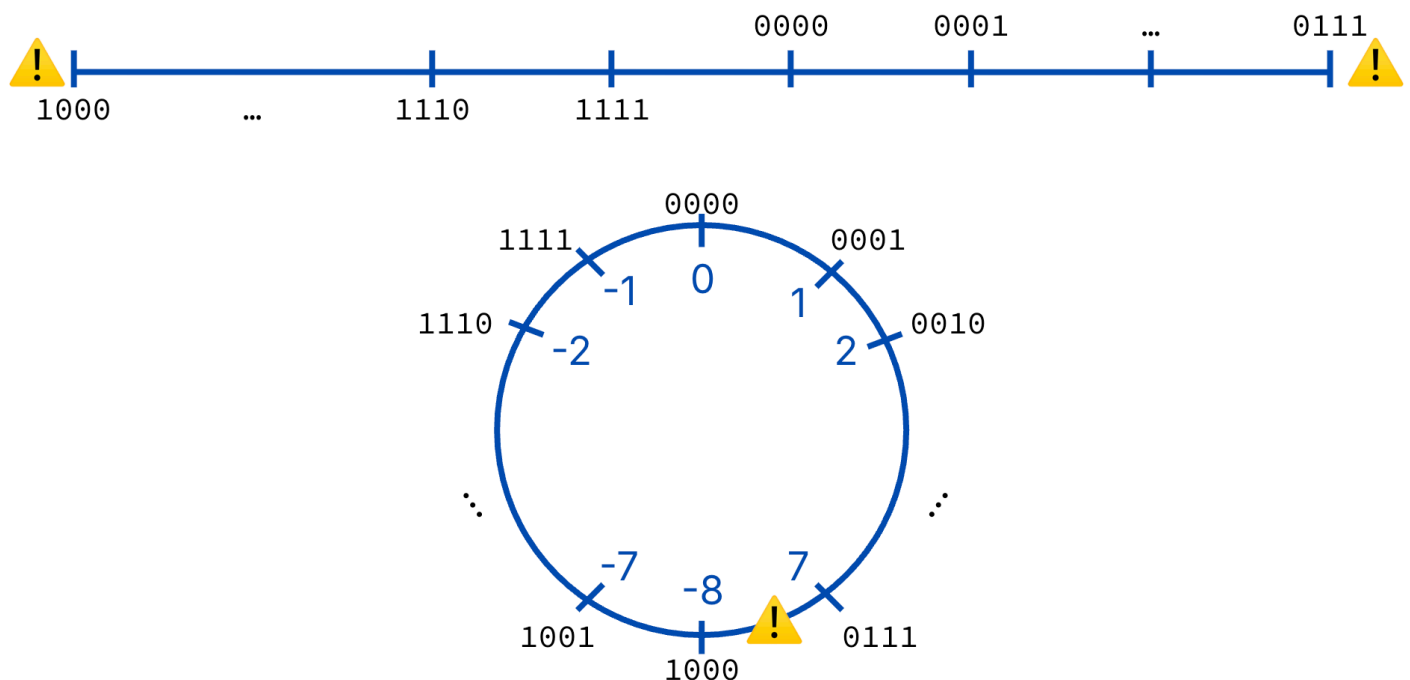


Figure 4: Top: A number line indicating where integer overflow occurs. Bottom: A number “wheel” indicating the same integer overflow location.

In [Figure 4](#), 0 through 7 stays the same as it has for every representation. But then it jumps to -8. That cool top-level term (-8) pulls all negative numbers down by **one** so there is no overlap at zero.

6 Two's Complement: C standard (as of 2025)

Two's complement is the C23 standard number representation for signed integers. Again, the built-in `int` is ambiguous because it does not specify bitwidth. And again, the header `stdint.h` accommodates typedefs like `int8_t`, `int16_t`, `int32_t`, etc., for signed integer representations.



L02 Number Representation
Integer Representations

L02 Number Representation
Summary

